モデル比較レポート統合版

AI組織のモデル選定 実測記録

2026年11月8日

この統合版は、2026年11月8日に AI組織の2つのプロジェクトで実施したモデル選定の実測レポートと、その背景を解説する図解資料を1つにまとめたものです。第1部・第2部の数値はすべて実測ファイルから生成しており、第3部は既存の図解資料を収録しています。その図解資料の導入部は、本統合版では全体の冒頭に「はじめに」として配置しています。

目次

はじめに　なぜエントロピー削減が必要か 2 ページ
エントロピー削減の仕組みがなぜ要るのか（総量が増え続ける理由と、その問題点）と、本レポートの構成

第1部　埋め込みモデル比較（AI組織 Issue #19「エントロピー削減の仕組み」V-EMB） 3 ページ
7モデルを同一条件で実測。採用：cl-nagoya/ruri-v3-310m

第2部　STTモデル比較（AI組織 Issue #11「面談準備・振り返りノート」V1） 5 ページ
faster-whisper 3モデルを実測。採用：large-v3-turbo

第3部　次元削減の図解（Issue #19 関連資料） 7 ページ
PCA の仕組みと、削減作業の一連のフロー図を収録

出典

第1部：entropy-reduction リポジトリ bench/v-emb/results.json／results_qwen.json（固定ペアセット v1、全て架空の文面）
第2部：stt-bench/results/ 配下のモデル別実測 JSON（faster-whisper、CER・処理時間・ピークメモリ）
第3部：既存資料「次元削減の図解」（2026年11月8日作成）を収録（同資料の導入部は、本統合版では冒頭の「はじめに」に配置）

はじめに：なぜエントロピー削減が必要か

モデル比較レポート統合版／2026年11月8日

本資料で解説するベクトル化・次元削減・比較・判定の一連の技術は、AI組織 Issue #19「エントロピー削減の仕組み」の部品である。技術の話に入る前に、その仕組みがなぜ要るのかを先に押さえておく。

なぜエントロピーは増え続けるのか

ルール・スキル・メモリなどの md 文書は、運用の中で「足す」操作が日常的に起きる。新しい決め事、例外、事故の教訓は、その都度どこかへ書き足されていく。一方で「減らす」操作は、誰かが意識して仕組みにしない限り起きない。この非対称があるため、放っておけば文書の総量は増える一方になるのが自然な帰結であり、総量の増加そのものがエントロピーの増加である。

なぜ増え続けることが問題か

密度の低下：読み手（エージェント）の注意力は有限で、文書の量が増えるほど1件あたりに割ける注意は薄まる。大事なルールほど、量の中に埋もれて目立たなくなる。

矛盾の増加：同じ内容が複数箇所に書かれていると、片方だけが更新されて内容が食い違う。箇所と組み合わせが増えるほど、衝突の機会も増えていく。

実質的な無効化：書いてあっても、探せない・読み切れないルールは「無い」のと同じである。しかも、例外や事故の教訓のような「めったに使わないが、決定的に大事な記述」ほど埋もれやすい。

静かな劣化：量の増加は少しずつ進むため、壊れていることに気づきにくい。判断のぶれや作業の漏れとして表面化して、初めて問題に気づくことになる。

本人が明言している通り、「長すぎる md ファイルは有害」である。長い文書は、読む側の解釈と注意力への依存を増やし、漏れ・解釈ブレ・矛盾を招く。

だから、「増やす仕組み」と同じくらい「測って、減らす仕組み」が要る。それが Issue #19 の仕組みで、運用は2つの分担で回す。週次の削減サイクルが「治療」として候補の仕分けと削減の実行を担い、月次の棚卸しが「健康診断」として検出と深い点検を担う。以下本文では、その削減サイクルの中核にある技術 ― 文章をベクトル化し、次元削減して比較し、判定し、意味が失われていないかを確認するまで ― を順に図解する。

本レポートの構成

本レポートは、この「測って、減らす仕組み」を支えるモデル選定の実測記録（第1部・第2部）と、その中核技術の図解（第3部）からなる。

- **第1部　埋め込みモデル比較**（AI組織 Issue #19 V-EMB）：エントロピー削減の候補検出（意味の重複の当たり付け）に使う埋め込みモデルを、固定ペアセットで7モデル実測し、選定した記録。採用：cl-nagoya/ruri-v3-310m。
- **第2部　STTモデル比較**（AI組織 Issue #11 V1）：面談の文字起こしに使う STT モデルを、faster-whisper の3モデルで実測し、選定した記録。採用：large-v3-turbo。
- **第3部　次元削減の図解**（Issue #19 関連資料）：削減サイクルの中核技術 ― ベクトル化、次元削減（PCA／SVD）、比較、判定、保存率テスト ― を図で解説する。

第1部　埋め込みモデル比較レポート

AI組織 Issue #19「エントロピー削減の仕組み」V-EMB（埋め込みモデル選定）／測定日：2026年11月8日／測定環境：Muse VM（CPU）実測

目的と測定条件

エントロピー削減の候補検出（意味の重複の当たり付け）に使う埋め込みモデルを、実測で選ぶための比較です。当初6モデルを測定し、同日に Qwen3-Embedding-0.6B を同一条件で追加実測しました（計7モデル）。固定ペアセット v1（全て架空の文面）を使い、言い換えペア12件・否定ペア12件・無関係ペア6件についてコサイン類似度を測りました。見るべき指標は「言い換えには高く、否定には低く出るか」、特に**分離度**（言い換えペアの類似度が否定ペアを上回る割合。1.0で完全分離）です。なお Qwen3-Embedding は本来、指示文（instruction）を付けて使う設計のモデルですが、本測定では他モデルと同じ「指示文なし」の素の条件で測っています。

ペア設計の意図：日本語の難所を測る

この固定ペアセットは、文章の「意味の近さ」を測るうえで日本語が特に難しい2点を、意図的に突く設計になっています。

- **否定**：日本語の否定は、文末のわずかな違い（〜できる／〜できない、〜てよい／〜てはいけない）で意味が正反対になります。ところが文の表層（使われている単語の重なり）はほぼ同じです。モデルが表層の似せに騙されず、意味の反転を捉えられるかを測ります。
- **言い換え**：逆に、語彙や文体が大きく変わっても意味は同じ、というペアです（例：「会議の資料は前日までに共有フォルダへ置いてください」／「ミーティングで使うファイルは、開催日の前日までに共有ドライブにアップしておいてください」）。表層が違っても意味の同一性を捉えられるかを測ります。

つまり分離度は、単なる類似度の平均値ではなく、**「日本語の難しい部分にどれだけうまく対応できるか」の指標**です。実測でも、この2点の明暗がはっきり分かれました。

- **表層に騙された例（e5-small）**：否定ペア「USBメモリの使用は許可されている」と「USBメモリの使用は許可されていない」に 0.989 を付けました。自モデルの言い換え平均（0.924）より高い値で、正反対の文を、言い換え同士よりも“似ている”と判定したことになります。
- **際どく競った例（ruri-v3-310m）**：最難の否定ペア「会議室は予約なしで使える」と「会議室は予約なしでは使えない」でも 0.945 で、言い換え平均（0.955）は下回りました。ただし言い換えの最小値（0.936）よりは上にあるため、言い換え×否定の全144通りの組み合わせのうち数件だけが僅差で逆転します。分離度が 1.0 ではなく 0.972 なのは、この数件分です。

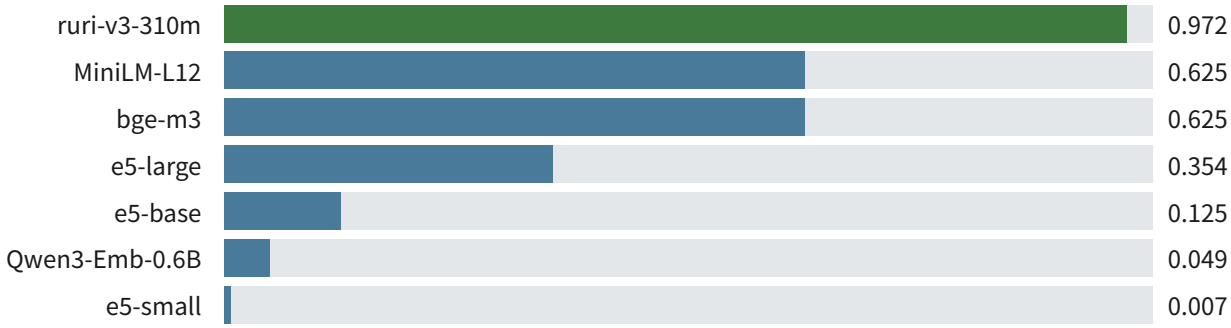
比較結果

モデル	次元	容量MB	文/秒	言い換え	否定	無関係	差	分離度
ruri-v3-310m	**768**	**1,267**	**7.3**	**0.955**	**0.921**	**0.761**	**+0.033**	**0.972**
MiniLM-L12	384	500	46.6	0.819	0.759	0.046	+0.059	0.625
bge-m3	1024	2,293	4.4	0.844	0.829	0.434	+0.015	0.625
e5-large	1024	2,262	3.7	0.949	0.959	0.795	-0.009	0.354
e5-base	768	1,134	11.8	0.939	0.967	0.798	-0.028	0.125

Qwen3-Emb-0.6B	1024	1,208	8.4	0.768	0.900	0.335	-0.132	0.049
e5-small	384	493	40.6	0.924	0.974	0.809	-0.050	0.007

「差」は言い換え平均 − 否定平均。マイナスは否定の方が高く出ている（＝区別できていない）ことを示します。

分離度（言い換え vs 否定）



読み取れること

- **ruri-v3-310m が分離度 0.972 で突出**。言い換え平均も最高（0.955）で、言い換えの最小値（0.936）も最も安定しています。日本語特化モデルの面目躍如です。
- **e5 系は否定の区別に失敗**。3モデルとも否定平均が言い換え平均を上回りました。最悪の例（e5-small）：「USBメモリの使用は許可されている」と「USBメモリの使用は許可されていない」の類似度が 0.989 ── 意味は正反対なのに、ほぼ同一と判定しています。
- **追加実測の Qwen3-Embedding-0.6B も、指示文なしの条件では否定を区別できませんでした**。言い換え 0.768 に対して否定 0.900 と逆転し、分離度は 0.049。最悪の否定例は「その情報は社外に公開できる」と「その情報は社外に公開できない」で 0.973 でした。指示文を付ける本来の使い方では評価が変わりうるものの、他モデルと同一条件の比較では ruri に及びません。
- **類似度の絶対値は全体に高い**。ruri でも無関係ペアで 0.761 あり、固定の閾値で線引きする運用はできません。候補の拾い方は順位・相対評価を軸にし、最終判定は判定担当が行う設計が必須であることを、実測が裏付けました。
- MiniLM は最速（46.6文/秒）ですが言い換えが弱く（0.819）、bge-m3 は全項目が中庸でした。速度が要る用途なら MiniLM、という位置づけです。

> **決定（2026年11月8日・本人）**：採用モデルは **cl-nagoya/ruri-v3-310m**。Qwen3-Embedding-0.6B の追加実測でも結論は変わらず、採用は ruri のままです。#19 の detect はこのモデルで実装済みです（モデル名は設定値として保持し、差し替え可能）。

出典：entropy-reduction リポジトリ bench/v-emb/results.json および results_qwen.json（本レポートの数値はこの実測ファイルから直接生成）。ペアセットと測定スクリプトも同ディレクトリに保管されています。

STTモデル比較レポート

面談文字起こし用モデルの選定（AI組織 Issue #11「面談準備・振り返りノート」V1）

作成日：2026年11月8日

1. 目的

面談の録音を文字起こしする工程（V1）で使う音声認識モデルを、実測に基づいて選定する。 候補は faster-whisper の medium、large-v3、large-v3-turbo の3モデルで、 いずれも Hugging Face から取得し、測定用VM上でローカル実行した（外部サービスへ音声は送らない）。

2. 測定条件

3モデルとも同一条件で測定した。エンジンは faster-whisper 1.2.1（ctranslate2 4.8.5）、 量子化は int8、デバイスは CPU（4スレッド）、言語は日本語（ja）、beam_size は 5、VAD フィルタは無効。

音声は正解テキスト付きの日本語音声（面談形式の台本の読み上げ）で、2系統に分かれている。 system（相手側を想定）は 64.87 秒、 mic（自分側を想定）は 116.53 秒、 合計 181.4 秒。測定環境は CPU 4コア／メモリ約7.7GB のVM。

CER（文字誤り率）は、認識結果と正解文を NFKC 正規化し空白と句読点を除いたうえで、 文字単位の編集距離を正解の文字数で割って算出した。ピークメモリは測定プロセスのピーク RSS。

3. 結果

モデル	CER system	CER mic	CER 合計	RTF	処理時間	読み込み	ピークメモリ
medium	0.48%	1.53%	1.16%	0.770	139.7 秒	17.7 秒	1,788MB
large-v3	0.48%	8.40%	5.65%	1.251	226.9 秒	23.4 秒	3,176MB
large-v3-turbo	**0.48%**	**0.51%**	**0.50%**	**0.703**	**127.5 秒**	**22.7 秒**	**1,623MB**

処理時間は音声合計181.4秒に対する文字起こし時間。トラック別の RTF は、 system：medium 0.825／large-v3 1.203／large-v3-turbo 0.660、 mic：medium 0.739／large-v3 1.278／large-v3-turbo 0.727。

4. モデルごとの所見

- **large-v3-turbo**：精度・速度・メモリの全てで3候補中最良。 mic の誤りは「賞与→省与」のような1文字の違いが中心で、合計 CER は 0.50% にとどまる。 ピークメモリも 1,623MB と最も小さく、共用VMでも余裕を持って動かせる。
- **medium**：合計 CER 1.16% で十分に実用的な精度。ただし mic では 「賞与が年2回→少女が歳2回」「承知いたしました→招致いたしました」のような 同音・似音の取り違えが目立つ。読み込みは17.7秒と最速だが、 速度とメモリは turbo とほぼ同等で、精度で劣るため採用する理由がない。
- **large-v3**：system は 0.48% と他モデルと同等だが、mic が 8.40% に悪化した。 内訳は、発話の一部が丸ごと抜ける脱落（正解文「出社日についても柔軟に対応できます。」が欠落）と、 末尾に音声にない一文（「ご視聴ありがとうございました」）が付く幻覚。 RTF も 1.251 と最も遅く、ピークメモリ 3,176MB は共用VM（約7.7GB）でも重い。

5. 指標の読み方

- **CER（文字誤り率）**：正解の文字数に対する、間違った文字（置換・脱落・余分な挿入）の割合。 低いほど正確で、0.50% はおおよそ200文字に1文字の誤り。
- **RTF（実時間係数）**：処理時間 ÷ 音声の長さ。1.0 で音声と同じ時間かかり、 0.703 なら60分の面談を約42分で処理できる。

6. 結論

選定結果は **large-v3-turbo** で、2026年11月8日に本人が確定した。 パイプラインには既定モデルとして組み込み済みで、実モデルと計測用音声による動作確認も終えている （音声181秒に対し全体約139秒で処理し、話者ラベル付きの文字起こしを生成）。

暫定の合格ライン（CER 10%以下／RTF 1.0以下、つまり処理時間は面談時間の1倍以内）と比べると、 large-v3-turbo は CER 0.50%・RTF 0.703 で余裕を持って満たす。 medium もライン内ではあるが精度で劣り、 large-v3 は RTF 1.251 でラインを超過する （60分の面談で約75分かかる計算）うえ、mic の CER 8.40% も劣後する。

なお、合格ラインの確定値（J-9）は、台本リハーサルの実測後に本人が最終確定する予定であり、 本レポートで用いたラインは暫定値である。

7. 出典

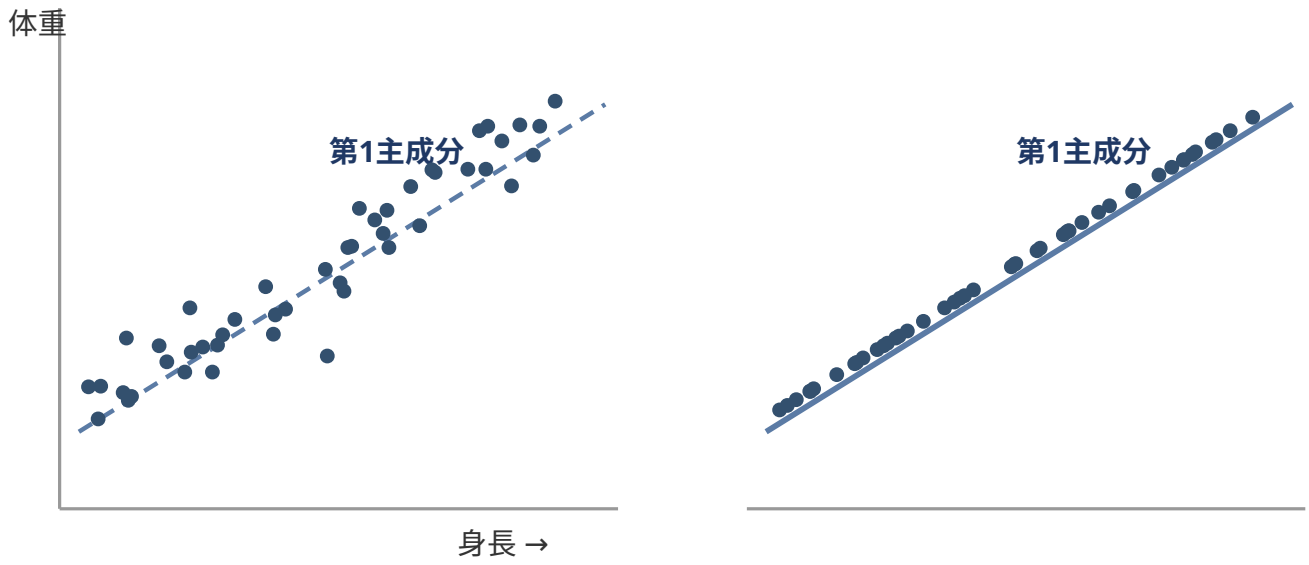
- 実測結果：stt-bench/results/medium__int8.json、large-v3__int8.json、large-v3-turbo__int8.json
- 計測スクリプト：stt-bench/scripts/bench_one.py、正解テキスト：stt-bench/audio/truth.json
- 確定の記録：interview-notes リポジトリ docs/開発ノート.md「V1 実測結果」、pipeline/SPEC.md「V1 追加契約」

次元削減（PCA）の図解

作成日：2026年11月8日（AI組織 Issue #19「エントロピー削減の仕組み」関連資料）

次元削減とは、たくさんの数字で表されたものを、情報の損失をできるだけ抑えながら少ない数字に圧縮すること。本資料では、その仕組み（PCA：主成分分析）を図で示し、削減作業の一連の流れに加えて、「ベクトルと文章の関係」と「比較の範囲・切り口」を補足し、最後に、架空のルール文で全工程をなぞる実例を示す。

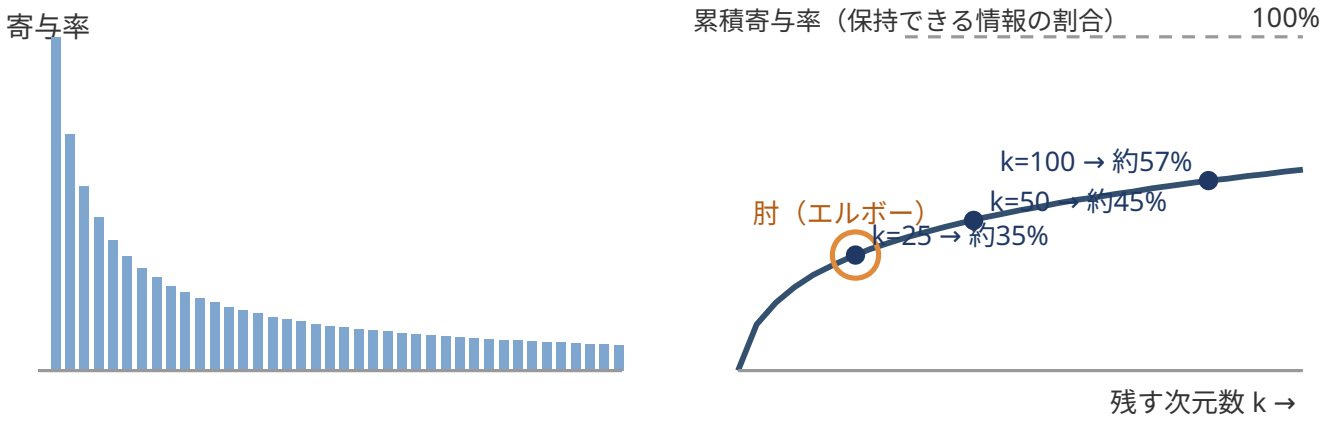
1. 点の雲を「線の上の位置」に落とす



元の姿：各点は2つの数字（身長・体重）　　投影後：全点が第1主成分の直線上に落ちる

点の雲が右上がりに伸びているのは、2つの数字が連動しているため（実質の情報はだいたい1方向に詰まっている）。雲が最も伸びている方向＝第1主成分の直線に全点を落とすと、各点は「線の上の位置」という1つの数字で表せるようになる。点同士の並び順と間隔はほぼそのまま残り、失われるのは「線からどれだけ外れているか」の分だけ。これが 2次元 → 1次元 の次元削減であり、PCA はこれを何百次元のデータでも同じ原理で行う。

2. 何次元まで残すか（寄与率とエルボー）



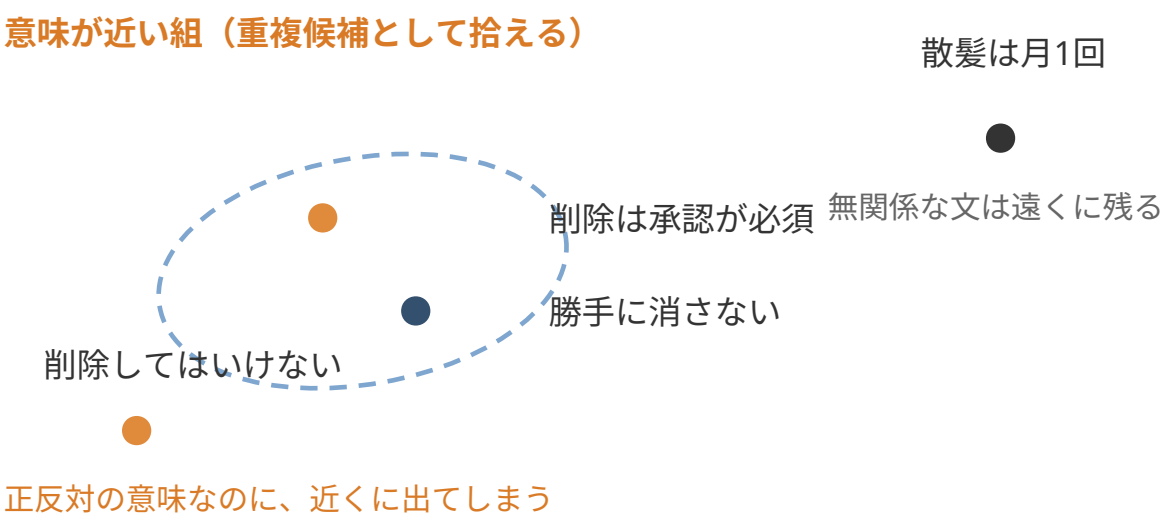
主成分ごとの寄与（大きい順・上位40個）

文章のベクトル（768次元）の場合、主成分は「情報の濃い順」に並んでいて、尻尾の方はほぼノイズ。累積寄与率の曲線は、はじめ急に立ち上がってから寝ていく。この折れ曲がりが肘（エルボー）で、肘を過ぎると次元を増や

しても保持できる情報はほとんど増えない。この例では k=25 で約35%、k=50 で約45%、k=100 で約57% を保持できる。つまり次元数は「何次元にしたいか」からではなく、保持できる情報量の側から決まる。

※ 本図の寄与データは減衰式 1/(i+2)^0.85 で生成した概念図であり、実測値ではない。

3. 文章に当てはめると



文を768個の数字（埋め込みベクトル）にし、数十個まで次元削減した後の「地図」では、意味の近い文同士が近くに残る。だから「削除は承認が必須」と「勝手に消さない」のような重複候補を、近さで機械的に拾うことができる。ただし「削除してはいけない」のような正反対の文も近くに出てしまうため、拾う工程と「本当に同じ意味か」を判定する工程は分ける必要がある。

4. 次元削減から、意味が失われていないかの確認まで

1. 元の文章
ルール・スキルなどの md 文書

▼

2. チャンク分け
文書を文・段落の塊（チャンク）に切り分ける。比較はこの塊の単位で行う

▼

3. ベクトル化（埋め込み）
チャンクごとに768個の数字に変換（Transformer系・ローカル実行・0円）

▼

4. 次元削減と比較
PCAで数十次元に圧縮し、類似ペア（重複候補）を検出

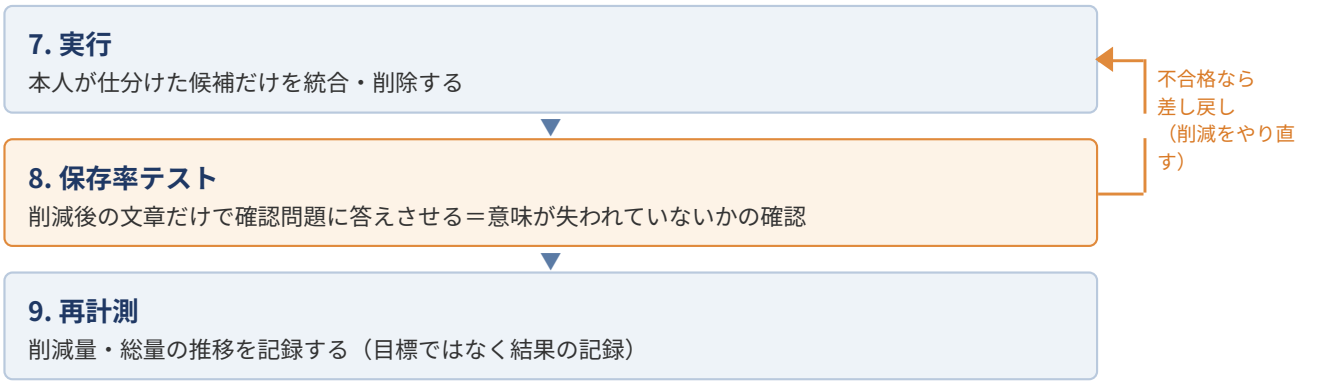
▼

5. 判定
本当に同じ意味か、否定・例外まで読んで判定する

▼

6. 仕分け
候補を台帳に提示し、本人が承認する

▼



削減は「似ているものを消す」だけでは終われない。実行の直後に置く保存率テストが「意味が失われていないかの確認」に当たる：あらかじめ用意した確認問題の固定セット（事故・例外系の問いを必ず含む）に、削減後の文章だけを根拠に答えさせ、元の意味が残っているかを検証する。不合格なら削減の実行へ差し戻しになる。なお、各段（計測・検出・判定・提示・実行・保存率テスト・再計測）はそれぞれ独立した単位として作り、単体でのテスト・差し替え・再実行ができるようにする。

4.5 チャンク同士をどう比較するか

第4節のフローは「チャンク分け → ベクトル化 → 次元削減と比較」と進んだ。ここでは、その「比較」が具体的にどう行われるかを図にする。例として、第7節の文A・文B・文Cに、第3節の「散髪は月1回」を加えた4つのチャンクを使う。

塊は表（DataFrame）になる

切り分けた塊は、1行＝1チャンクの表に並べる。列は「塊ID／出典／原文テキスト／ベクトル（数値の並び）」で、この表が比較の材料のすべてになる。

塊ID	出典	原文テキスト	ベクトル（数値の並び）
K1	文A（スキル文書）	スプレッドシートを編集したら、必ず読み戻して見切れがないか確認する	`[0.12, −0.45, 0.88, 0.03, …]`
K2	文B（ルール集）	シートを変更した後は読み戻し検証を行い、表示が切れていないことを確認する	`[0.10, −0.41, 0.85, 0.05, …]`
K3	文C（ルール集）	読み戻し確認は、自動生成されただけのシートでは省略してよい	`[0.31, −0.22, 0.79, −0.12, …]`
K4	メモリ	散髪は月1回	`[−0.62, 0.18, −0.05, 0.44, …]`

原文テキストの列は、類似度の計算には使わない（計算の対象はベクトルの列だけ）。判定係が「本当に同じ意味か」を確かめるときに、原文同士を読み比べるために保持しておく列である。

類似度の総当たり表（行列）

次に、圧縮したベクトル同士で類似度を総当たりで計算し、行列の形に並べる。自分同士（対角）は比べないので対象外、表は対称で、K1とK2の値は K2とK1 でも同じになる。

	K1（文A）	K2（文B）	K3（文C）	K4（散髪）
K1（文A）	—	**0.93**	0.81	0.21
K2（文B）	**0.93**	—	0.78	0.18

K3（文C）	0.81	0.78	—	0.24
K4（散髪）	0.21	0.18	0.24	—

数値は概念例。K1–K2 は第7節の言い換え重複で 0.93、K1–K3 は例外を含むペアで 0.81。

候補を抜き、原文で判定する

総当たり表ができる
塊×塊の類似度が並ぶ

→

順位の高いペアを抜く
重複候補として抜き出す（例：K1–K2 = 0.93）

→

原文で最終判定する
判定係がベクトルではなく原文テキスト同士を読んで、統合する／しないを決める

行列の中で順位の高いペアを重複候補として抜く。ただし、抜いた時点ではまだ「当たり」の段階で、統合してよいかは決まっていない。K1–K3（0.81）のように「そこそこ近い」ペアが例外を含む別ルールとして判定で外れるのは、この段階である（第7節の実例）。

組み合わせの数と、次元削減の意味

塊が N 個あれば、比べる組み合わせは N×(N−1)/2 通りになる。塊が1,000個なら約50万通りで、しかも1ペアあたりの計算はベクトルの次元数に比例して重くなる。だから総当たりの前に次元削減で数十次元まで圧縮しておく（第2節）。次元削減は、この総当たりを現実的な計算量に収めるための工程でもある。

5. ベクトルは文章に戻さない

元の文章
本文（読み書きの対象）

ベクトル化 →

ベクトル（数字の列）
比較専用の控え＝索引

類似度で比較 →

似たペア
重複候補の当たり

×

ベクトルから文章は作らない（復元できない）

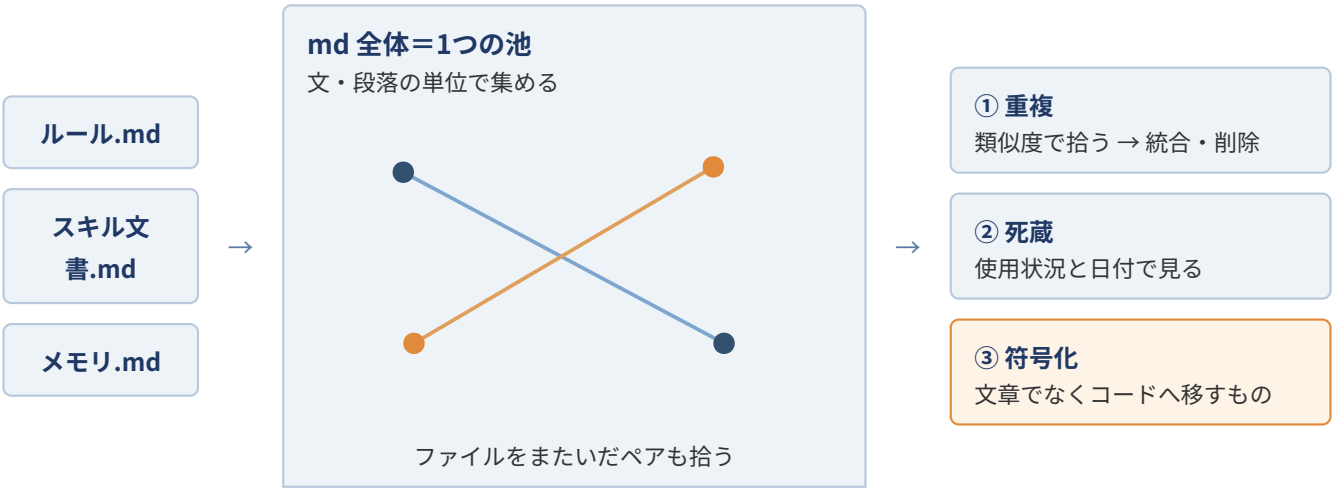
削減作業は文章そのものへ
消す・統合する・コードへ移す
当たりをもとに、文章そのものを編集する

保存率テストも文章で
ベクトルではなく削減後の文章を使う

ベクトル化は比較のためにだけ行う内部処理で、ベクトルから元の文章に戻す工程はない（埋め込みベクトルからは原文を復元できない）。ベクトルの役割は、「どの文とどの文が似ているか」の当たりを付けることまで。実際の削減作業（消す・統合する・コードへ移す）は、原文の文章そのものに対して行い、意味が残っているかの確認（保存率テスト）も、ベクトルではなく削減後の文章そのものを使う。

たとえると、ベクトルは本の「索引」のようなもの。索引で当たりを付けて、読み書きは本文で行う。索引から本文を作ることはできないし、作る必要もない。

6. 比較の範囲と3つの切り口



比較は1つの md ファイルの中だけで完結するものではない。対象の md 全部（ルール・スキル・メモリなど）を1つの池として集め、文・段落の単位に分けて、全体の中で似たペアを探す。同じファイル内の重複も拾うが、実害が大きいのはファイルをまたいだ重複（同じルールが複数の場所に書かれている状態）である。

削減候補の切り口は3つある。①重複は類似度で拾い、統合・削除の対象にする。②死蔵は、長く使われていない・参照されていないものを、使用状況と日付から見る（似ているかどうかでは分からない）。③符号化は、文章の形で置いておくよりコード（ハーネスや tool）に移した方がよいものを、判定の段階で分ける。

切り口ごとの抜き方

重複の抜き方：4.5節の類似度の総当たり表から、順位の高いペアを重複候補として抜く。使うのはベクトル同士の類似度だけである。

死蔵の抜き方：ベクトルは使わない。対象ごとに「最終使用日」と「参照元の数」を集計し、参照元が無いもの、最終使用日が境界の日数（設定値）以上古いものを候補として抜く。境界の判定は「以上」で統一し、境界ちょうどのものも候補に入れる。この集計は月次棚卸し側のインベントリ（肥大・死蔵の集計）が担い、その結果が候補の入力になる。

符号化の抜き方：数値では判定しない。文章で書かれたルールのうち、手順が機械的・反復的で、文章のまま置くよりコード（ハーネスや tool）に固定すべきものを、判定する側が原文を読んで抜く。

どの切り口で抜いた候補にも、判定の証拠3点（最終使用日／参照元／影響範囲）を付けてから、判定の段へ進める。証拠が揃わない候補は「要確認」として分ける。

7. 文章の実例で全工程をなぞる

最後に、架空のルール文3つを使って、第4部の流れを1件まるごとなぞってみる。文Aと文Bは、別のファイルに書かれた言い換えの重複。文Cは似て見えるが例外を含む別のルールで、ここが判定と保存率テストの見せ場になる。

登場する文（別々のファイルに書かれた想定・架空のルール文）

文A（スキル文書）	「スプレッドシートを編集したら、必ず読み戻して見切れがないか確認する」
文B（ルール集）	「シートを変更した後は読み戻し検証を行い、表示が切れていないことを確認する」
文C（ルール集）	「読み戻し確認は、自動生成されただけのシートでは省略してよい」

1. チャンク分けとベクトル化
文A〜Cは1文がそのまま1チャンクになる。各チャンクを768個の数字の列に変換する
文A → [0.12, −0.45, 0.88, 0.03, …]（数字の抜粋は例）

▼

2. 次元削減と比較
圧縮したベクトル同士で類似度を計算する（数値は概念例）
A–B：0.93（高い） ／ A–C：0.81（そこそこ近い）
→ 近いペアは拾えるが、類似度だけでは統合してよいか決められない

▼

3. 判定
AとBは言い換えで同じルール → 統合候補にする
Cは例外（省略してよい）を含む別のルール → 統合しない

▼

4. 仕分け
候補を台帳に提示し、本人が承認する

▼

5. 実行
Bを削除して正本をAに一本化し、B側には参照だけ残す
削減量は結果として記録するだけで、目標ではない

▼

6. 保存率テスト
削減後の文章だけを根拠に、確認問題へ答えさせる
Q1「シートを編集した後、何を確認する？」→ 統合後も答えられる
Q2「読み戻し確認を省略できる場合はある？」
→ C（例外）が消えていたら答えられなくなる＝例外が守られたかの検査
不合格なら実行へ差し戻しになる

▼

7. 再計測
削減前後の総量を記録して、一巡の終わり

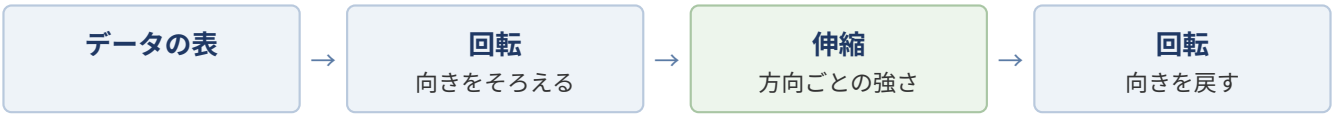
肝は2か所ある。1つ目は、文Cも類似度0.81と「そこそこ近い」数字で出てくるため、類似度だけに任せると例外ルールを誤って消してしまうこと。判定の工程で否定・例外まで読むのはそのためである。2つ目は、保存率テストの Q2 のように、例外が残っているかを確認問題で確かめられること。削減後の文章だけで全問に答えられることが、「意味が失われていない」ことの担保になる。

※ ベクトルの数字の抜粋と類似度（0.93／0.81）は概念例であり、実測値ではない。

8. SVD（特異値分解）と PCA の関係

ここまでの説明では「次元削減＝PCA」と書いてきましたが、実際に計算を担っているのは SVD（特異値分解）という別の道具です。関係をひと言で言うと、PCA は「目的と手順」、SVD は「計算の道具」です。

SVD：どんな表（行列）でも、この3操作の組み合わせに分解できる



伸縮の強い方向だけを残すと、情報を保ったまま次元を落とせる（切り捨て SVD）

SVD は、どんな行列（データの表）でも「回転 × 伸縮 × 回転」の3つの操作の組み合わせに分解できる、という汎用の数学の道具です。伸縮の度合い（特異値）が大きい方向ほど、そのデータの「情報の濃い方向」です。強い方向だけを残して弱い方向を捨てるのが切り捨て SVD で、これが次元削減の実体になります。

PCA との違いは「中心化」だけ

通常の PCA は、最初にデータの各行から平均を引く（中心化：点の集まりの重心を原点に引っ越す操作）を行い、その中心化したデータに SVD をかけて主成分を取り出します。つまり PCA の中身は「中心化つきの SVD」であり、両者の実質的な違いは中心化をするかどうか、ただ1点です。中心化は「散らばりの方向」を純粋に見るためには必要な手順ですが、用途によっては裏目に出ます。

実測エピソード（Issue #19 の実装で実際に起きたこと）

埋め込みベクトルを「中心化する通常の PCA」で圧縮したところ、コサイン類似度が崩壊しました。言い換えペアの類似度は 0.943 から 0.253 へ落ち、重複候補は0件になりました。理由は、コサイン類似度が「原点から見た角度」で測る量だからです。中心化は原点そのものを引っ越す操作なので、全ベクトルの角度が組み替わってしまったのです。

そこで中心化しない切り捨て SVD に替えたところ、圧縮の前後で順位精度（分離度）0.972 が不変であることを実測で確認できました。PCA が劣っているのではなく、「圧縮して、そのままコサイン類似度で比べる」という目的と測り方の組み合わせでは、中心化しない SVD が合っていた、というのが結論です。

本資料の読み替え注記：2.（寄与率とエルボー）で述べた「情報の濃い順に方向が並ぶ」考え方は、SVD の特異値の並びとそのまま対応します。また 4. と 7. のフロー図で「PCA で圧縮」と書いた段（「次元削減と比較」）は、道具としては SVD で実行できる工程であり、中心化するかしないかは用途に応じて選べる、と読み替えてください。Issue #19 の実装では、上記エピソードの通り中心化しない切り捨て SVD を採用しています。

出典：AI組織 Issue #19「エントロピー削減の仕組み」P2 完了報告（GitHub Issue #19 コメント）。数値は同報告に記録された実測値です。

付録：分離度を詳しく読む

AI組織 Issue #19「エントロピー削減の仕組み」V-EMB（埋め込みモデル選定）補足資料／実測値の出典は entropy-reduction リポジトリ bench/v-emb/results.json・results_qwen.json（固定ペアセット v1、測定日：2026年11月8日）

1. 分離度とは何か（定義）

分離度は、言い換えペア12件と否定ペア12件の類似度を総当たりで比べ、**「言い換えの類似度が否定の類似度を上回る」組み合わせの割合**です。組み合わせは 12 × 12 = 144 通り。1.0 なら全ての組み合わせで順序が正しい（完全分離）、0.5 なら偶然と同程度、0 に近いほど「正反対の文ほど似ている」と判定していることになります。類似度の絶対値ではなく**順序の正確さ**を測る指標です。

採用した ruri-v3-310m の分離度 0.972 は、144 通り中 140 通りで順序が正しく、逆転は 4 通りだけ、という意味です。各モデルの逆転件数に直すと、差はよりはっきりします。

モデル	分離度	正しい順序	逆転	言い換え平均	否定平均	差（マージン）
ruri-v3-310m	**0.972**	**140 件**	**4 件**	**0.955**	**0.921**	**+0.033**
MiniLM-L12	0.625	90 件	54 件	0.819	0.759	+0.060
bge-m3	0.625	90 件	54 件	0.844	0.829	+0.015
e5-large	0.354	51 件	93 件	0.949	0.959	−0.010
e5-base	0.125	18 件	126 件	0.939	0.967	−0.028
Qwen3-Embedding-0.6B	0.049	7 件	137 件	0.768	0.900	−0.132
e5-small	0.007	1 件	143 件	0.924	0.974	−0.050

e5-small は 144 通り中 143 通りが逆転しており、「否定の文ほど似ている」とほぼ一貫して判定しています。平均値だけ見れば言い換え 0.924・否定 0.974 とどちらも「高い類似度」に見えますが、順序はほぼ全滅です。

2. なぜ「平均の差（マージン）」では足りないか

一見すると、言い換え平均 − 否定平均の「差」が大きければ区別できているように思えます。ところが実測では、**差が最大級だった MiniLM-L12（+0.060）が、分離度では 0.625 しか取れていません。** ruri の差は +0.033 と MiniLM の約半分なのに、分離度は 0.972 です。

理由は分布の広がり（ばらつき）にあります。MiniLM は類似度の標準偏差が 0.10〜0.14 と広く、平均では離れていても個別のペア同士では大きく重なります。一方 ruri は標準偏差が 0.011〜0.014 と極めて狭く、平均の差が小さくても分布がほぼ重なりません。

重複候補の検出は「平均」ではなく「1 ペアごと」の判断の積み重ねです。だからモデルの良し悪しは、平均の差ではなく、**個別のペアで順序を間違える頻度**で測る必要があります。分離度はその頻度をそのまま数えた指標です。

3. 逆転はどこで起きるか — ruri の 4 件

ruri で逆転した 4 件は、いずれも「最も難しい否定」と「最も弱い言い換え」がぶつかる僅差のゾーンです。最難の否定ペア「会議室は予約なしで使える／予約なしでは使えない」は 0.945、最も弱い言い換えペアは 0.936 で、その差はわずか 0.009。この帯域では、モデルの優劣ではなく測定の誤差範囲で順序が入れ替わります。

ここから設計上の帰結が 2 つ出ます。

- **固定の閾値で自動的に統合してはいけない。**類似度が高い帯域にも、正反対のペアが紛れ込む場所が残ります。
- **埋め込みの役割は「候補の当たり付け」まで。**#19 の設計（候補を出し、削減係が証拠つきで判定し、最終決定は本人の仕分け）になっているのは、この 4 件のような僅差ゾーンを人の判断に残すためです。分離度 0.972 は「自動で決めてよい水準」ではなく「当たり付けとして十分に信頼できる水準」と読みます。

4. 分離度のもう一つの仕事 — 圧縮の検収

分離度は、モデル選定だけでなく、次元削減（圧縮）の受け入れ検査にも同じ物差しとして使いました。中心化する通常の PCA で圧縮したところ、言い換えペアの類似度が 0.943 から 0.253 へ崩壊し、重複候補は 0 件になりました。中心化しない切り捨て SVD に替えたところ、**圧縮の前後で分離度 0.972 が不変**であることを実測で確認できました。

圧縮で壊れやすいのは類似度の絶対値ではなく順序です。だから検収も絶対値ではなく順序（分離度）で見る。選定と検収で物差しを統一したことで、「圧縮しても判定の質は落ちていない」と一貫した基準で言えるようになっています。

5. 限界と注意

- ペアセット v1 は架空文 30 件（言い換え 12・否定 12・無関係 6）の固定セットです。分離度はモデル単体の性質ではなく「モデル × ペアセット」の値であり、別の文面では変わりえます。
- 類似度の絶対値は全体に高く、ruri でも無関係ペアで 0.761 あります。分離度が高くても、固定閾値での自動判定はできません（第 3 節の通り）。
- Qwen3-Embedding-0.6B は本来、指示文を付けて使う設計のモデルです。本測定は他モデルと同一条件にするため指示文なしで測っており、本来の使い方では評価が変わりうる点に注意してください。